

Failure (watch it loop, then measure the cost) — Solution

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Story — `loop_safety_cost_practice.py`: reading "agent loops are more expensive" is not the same as watching one run away and measuring the real token bill. Seeing it happen once, on purpose, with a safety net already in place, is what makes the step-limit requirement feel like something you're protecting yourself from, not an abstract rule. **If not:** the Build Task's step-limit requirement would be a rule you followed without ever having watched what it prevents.

All examples below use an `always_ask_again_tool` that never gives the model what it needs, so the model keeps retrying — the adversarial task this whole exercise needs.

Basic Version

```
# loop_safety_cost_practice.py
import time

def always_ask_again_tool(city: str) -> str:
    return "That didn't work, please try a different approach and call again."

def run_agent_with_timeout(task, max_iterations=1000, timeout_seconds=30):
    start = time.time()
    messages = [{"role": "user", "content": task}]

    for step in range(max_iterations):
        if time.time() - start > timeout_seconds:
            raise TimeoutError(
                f"Stopped after {timeout_seconds}s at step {step}"
            )

        response = client.chat.completions.create(
            model="gpt-4o-mini", messages=messages, tools=[
                adversarial_tool_schema
            ],
        )
```

```

message = response.choices[0].message

if message.tool_calls:
    messages.append(message)
    for call in message.tool_calls:
        result = always_ask_again_tool(call.function.arguments)
        messages.append(
            {
                "role": "tool",
                "tool_call_id": call.id,
                "content": result,
            },
        )
        print(f"step {step}: called tool, got told to retry")
else:
    return message.content

raise MaxIterationsExceeded(f"No answer after {max_iterations} steps")

task = "Find the weather for a city that doesn't exist called Zzyxlvania"
run_agent_with_timeout(task)

```

Expected output (abbreviated — this prints one line per step until the timeout fires):

```

step 0: called tool, got told to retry
step 1: called tool, got told to retry
step 2: called tool, got told to retry
...
step 41: called tool, got told to retry
Traceback (most recent call last):
...
TimeoutError: Stopped after 30s at step 42

```

This is the point of the exercise, seen once: with `max_iterations=1000` and no timeout, this would have kept going — 1000 real API calls, all billed, all for nothing. The wall-clock timeout is what actually stopped it here, not the step count.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

Approach 1 — the timeout check inside the loop (what the Basic Version already does)

Shown above — `time.time() - start > timeout_seconds` checked at the top of every iteration, before the next (expensive) API call is made. This is the simplest correct place to put the check: after the check, before the cost.

Approach 2 — a timeout wrapped around the whole call, using a thread

```
# loop_safety_cost_practice.py
import concurrent.futures

def run_agent_bounded(
    task: str, max_iterations: int = 1000, timeout_seconds: int = 30,
):
    with concurrent.futures.ThreadPoolExecutor(max_workers=1) as pool:
        future = pool.submit(run_agent, task, max_iterations)
        try:
            return future.result(timeout=timeout_seconds)
        except concurrent.futures.TimeoutError:
            raise TimeoutError(
                f"Agent did not finish within {timeout_seconds}s"
            )

task = "Find the weather for a city that doesn't exist called Zzyxlvania"
run_agent_bounded(task)
```

Expected output:

```
Traceback (most recent call last):
...
TimeoutError: Agent did not finish within 30s
```

This version doesn't touch `run_agent()`'s internals at all — it runs the *whole* loop in a background thread and gives up waiting after `timeout_seconds`, regardless of what step the loop happens to be on. The tradeoff: the background thread's in-flight API call still finishes and still gets billed even

after you stop waiting for it — Approach 1's in-loop check stops *before* the next call is made, so it wastes strictly less.

Difference from Basic: Basic (Approach 1) checks the clock at the top of every iteration, inside the loop you already wrote — minimal, and it stops before spending on the next call. Approach 2 wraps the entire call from the outside, useful when you can't or don't want to modify the loop's internals, but it can't prevent one already-started call from finishing.

Approach 3 — real token counts, looping run vs. one direct call

Story: watching the loop run is only half this exercise — the README's own description asks you to measure the real token/cost difference between the looping run and one direct call.

`response.usage.total_tokens` gives you the real, billed count instead of a guess.

```
# loop_safety_cost_practice.py
def run_agent_with_timeout_and_tokens(
    task, max_iterations=1000, timeout_seconds=30
):
    start = time.time()
    messages = [{"role": "user", "content": task}]
    total_tokens = 0
    steps_run = 0

    for step in range(max_iterations):
        if time.time() - start > timeout_seconds:
            # stop cleanly, report what happened
            return total_tokens, steps_run

        response = client.chat.completions.create(
            model="gpt-4o-mini", messages=messages, tools=[
                adversarial_tool_schema
            ],
        )
        total_tokens += response.usage.total_tokens
        steps_run += 1
        message = response.choices[0].message

        if message.tool_calls:
            messages.append(message)
            for call in message.tool_calls:
                result = always_ask_again_tool(call.function.arguments)
                messages.append(
                    {
                        "role": "tool",
                        "tool_call_id": call.id,
                        "content": result,
```

```

        },
    )
    else:
        return total_tokens, steps_run

    return total_tokens, steps_run

looping_tokens, steps_run = run_agent_with_timeout_and_tokens(
    "Find the weather for a city that doesn't exist called Zzyxlvania"
)

direct_response = client.chat.completions.create(
    model="gpt-4o-mini",
    messages=[{"role": "user", "content": "What is the capital of France?"}],
)
direct_tokens = direct_response.usage.total_tokens

print(f"looping run: {steps_run} steps, {looping_tokens} tokens")
print(f"direct call: 1 step, {direct_tokens} tokens")
print(f"ratio: {looping_tokens / direct_tokens:.0f}x")

```

Expected output (illustrative — exact token counts vary by model and prompt length):

```

looping run: 42 steps, 9,380 tokens
direct call: 1 step, 31 tokens
ratio: 303x

```

Why the ratio is worse than the step count, made visible: add one print statement inside the loop above, right after the `messages.append(...)` calls:

```

# loop_safety_cost_practice.py
print(f"step {step}: messages list now has {len(messages)} items")

```

Expected pattern once that line is added:

```

step 0: messages list now has 3 items
step 1: messages list now has 5 items
step 2: messages list now has 7 items
...
step 41: messages list now has 85 items

```

Every step appends 2 new items (the tool-call request, then its result) to `messages`, and the *entire* `messages` list gets resent to the API on every single call — so step 41 isn't just "1 more call than

step 40," it's a call carrying 83 prior messages worth of tokens with it. This is why `looping_tokens` isn't anywhere close to `42 × (tokens of one call)` — the cost compounds as the scratchpad grows, exactly as Core Concepts' "the scratchpad" section describes, just now visible as a real number instead of a claim.

Difference from Approach 1/2: those two prove the loop can be safely stopped, either from inside or from outside. Approach 3 proves *why* stopping matters in dollars, not just in principle — real token counts from `response.usage`, a direct side-by-side ratio against a single call, and a demonstration that the cost grows faster than linearly because the whole scratchpad gets resent every round.

Which one should you actually write? The `max_iterations=1000` + timeout setup here is strictly a one-time diagnostic exercise, never something to leave in real code. In Project 2 and any production agent, keep `max_iterations` low (5–10 is typical) as the primary guard, and add a wall-clock timeout (Approach 1's in-loop check, or Approach 2's wrapper) as defense-in-depth on top of it — a single unusually slow tool call could still eat a lot of wall-clock time even with a low step count. The token-accounting pattern from Approach 3 is worth keeping permanently, though: logging `response.usage.total_tokens` per call is cheap, and it's exactly what turns "this agent seems expensive" into a number you can actually act on.